DWS — DESARROLLO WEB EN ENTORNO SERVIDOR

UD2: Introducción a PHP y Desarrollo Web en Servidor

Sintaxis fundamental, estructuras de control, formularios web, cookies, sesiones y seguridad en servidor

Bloque 1: ¿Qué es PHP y Arquitectura Servidor

APARTADOS §1 A §3

1. ¿Qué es PHP?

PHP es un lenguaje de programación especialmente utilizado para desarrollar aplicaciones web del lado del servidor.

Cuando utilizamos HTML, el navegador puede interpretar directamente el código:

```
<h1>Hola</h1>
```

Pero PHP funciona de otra manera.

El código PHP se ejecuta en el **servidor** y el resultado que recibe el navegador normalmente es HTML.

Por ejemplo:

```
<?php

echo "Hola mundo";
```

El servidor ejecuta PHP y genera:

```
Hola mundo
```

El navegador recibe el resultado, no el código PHP original.

1.1. Cliente y servidor

En una aplicación web intervienen normalmente dos partes:



Por ejemplo, cuando visitamos:

```
http://localhost/index.php
```

ocurre algo parecido a:

1. El navegador solicita `index.php`.
2. El servidor recibe la petición.
3. PHP ejecuta el código.
4. PHP genera una respuesta.
5. El servidor devuelve la respuesta.
6. El navegador muestra el HTML.

1.2. PHP no se ejecuta en el navegador

Este código:

```
<?php

echo "Hola";
```

no se ejecuta directamente en Chrome, Firefox o Edge.

Necesitamos un servidor con PHP.

Durante la unidad podremos utilizar, por ejemplo XAMPP.

---

2. Nuestro primer programa PHP

Creamos un archivo:

```
index.php
```

Y escribimos:

```
<?php

echo "Hola mundo";
```

La etiqueta:

```
<?php
```

indica el comienzo del código PHP.

La instrucción:

```
echo
```

sirve para enviar contenido a la salida.

Microejercicio 1 — Hola PHP

BÁSICO

Crea un archivo `index.php` que muestre:

```
Hola, soy [tu nombre] y estoy aprendiendo PHP.
```

Después modifica el programa para mostrar tres líneas:

```
Nombre: ...
Curso: ...
Lenguaje: PHP
```

Puedes utilizar varios `echo`.

Ver solución explicada y código PHP ▸

---

3. Comentarios

Los comentarios sirven para escribir información destinada a las personas que programan, no al ordenador.

Comentario de una línea:

```
// Esto es un comentario
```

También:

```
# Esto también es un comentario
```

Comentario de varias líneas:

```
/*
    Este comentario
    ocupa varias líneas.
*/
```

Ejemplo:

```
<?php

// Mostramos un mensaje
echo "Hola";
```

Los comentarios son importantes para explicar decisiones o partes complejas del programa.

No debemos llenar el código de comentarios innecesarios.

Bloque 2: Variables, Tipos y Operadores

APARTADOS §4 A §8

4. Variables

Una variable permite almacenar información.

En PHP las variables comienzan con `$`.

```
$nombre = "Ana";
$edad = 20;
```

Después podemos utilizar esas variables:

```
echo $nombre;
echo $edad;
```

También podemos combinar texto y variables.

```
$nombre = "Ana";

echo "Hola $nombre";
```

PHP sustituirá `$nombre` por su valor.

También podemos concatenar utilizando `.`:

```
echo "Hola " . $nombre;
```

4.1. Las variables pueden cambiar

```
$edad = 20;

echo $edad;

$edad = 21;

echo $edad;
```

Una variable puede recibir un nuevo valor.

4.2. Nombres de variables

Correcto:

```
$nombre;
$edad;
$precioProducto;
```

No debemos utilizar:

```
$nombre usuario;
$1nombre;
```

Una convención habitual es utilizar **camelCase**:

```
$nombreCompleto;
$precioFinal;
$numeroProductos;
```

Microejercicio 2 — Presentación

BÁSICO

Crea las variables:

```
$nombre
$apellidos
$edad
$curso
```

Y muestra una frase como:

```
Me llamo Ana García, tengo 20 años y estoy estudiando 2º DAW.
```

Ver solución explicada y código PHP ▸

5. Tipos de datos

PHP trabaja con diferentes tipos de datos.

Los más importantes al comenzar son:

Tipo	Ejemplo
`string`	`"Hola"`
`int`	`20`
`float`	`19.95`
`bool`	`true`
`null`	`null`
`array`	`["rojo", "verde"]`

Ejemplo:

```
$nombre = "Ana";
$edad = 20;
$precio = 19.95;
$activo = true;
$direccion = null;
```

Podemos utilizar `var_dump()` para conocer el tipo y el valor:

```
var_dump($nombre);
var_dump($edad);
var_dump($precio);
var_dump($activo);
```

Microejercicio 3 — Detectando tipos

BÁSICO

Crea cinco variables, una de cada tipo:

- string
- int
- float
- bool
- null

Utiliza `var_dump()` para comprobar su tipo.

Ver solución explicada y código PHP ▸

---

6. Constantes

Una constante representa un valor que no debería cambiar.

Podemos definirla mediante `const`:

```
const IVA = 0.21;
```

Después:

```
echo IVA;
```

Por ejemplo:

```
const NOMBRE_TIENDA = "DAW Shop";

echo NOMBRE_TIENDA;
```

Por convención, las constantes suelen escribirse en mayúsculas.

---

7. Operadores

Los operadores permiten realizar operaciones.

7.1. Operadores matemáticos

```
$suma = 10 + 5;
$resta = 10 - 5;
$multiplicacion = 10 * 5;
$division = 10 / 5;
$resto = 10 % 3;
```

El operador `%` obtiene el resto de una división.

```
10 % 3
```

produce:

```
1
```

7.2. Asignación

```
$edad = 20;
```

También podemos utilizar:

```
$contador += 1;
$contador -= 1;
$contador *= 2;
$contador /= 2;
```

Por ejemplo:

```
$contador = 0;

$contador += 1;
$contador += 1;
```

Ahora:

```
$contador = 2
```

---

8. Comparaciones

Podemos comparar valores:

```
$edad > 18
$edad < 18
$edad ⩾ 18
$edad ⩽ 18
$edad = 18
$edad == 18
$edad ≠ 18
$edad ≠= 18
```

= frente a ==

Esta diferencia es especialmente importante en PHP.

```
"5" = 5
```

es verdadero porque PHP puede realizar una conversión de tipos.

Sin embargo:

```
"5" == 5
```

es falso porque los tipos son diferentes.

En código moderno debemos preferir normalmente:

```
==
```

y:

```
≠=
```

cuando queremos comprobar también el tipo.

8.1. Operadores lógicos

Podemos combinar condiciones:

```
&&
||
!
```

Ejemplo:

```
$edad ⩾ 18 && $tieneCarnet
```

Significa:

> Tiene al menos 18 años Y tiene carnet.

Otro ejemplo:

```
$esAdmin || $esEditor
```

Significa:

> Es administrador O es editor.

Microejercicio 4 — ¿Es mayor de edad?

BÁSICO

Crea una variable:

```
$edad = 17;
```

Muestra:

```
Es mayor de edad
```

si tiene 18 años o más.

Prueba después con diferentes edades.

Ver solución explicada y código PHP ▸

Bloque 3: Estructuras de Control e Iteración

APARTADOS §9 A §11

9. Estructuras condicionales

Una condición permite ejecutar código dependiendo de una situación.

```
if ($edad ⩾ 18) {
    echo "Mayor de edad";
}
```

También podemos utilizar `else`:

```
if ($edad ⩾ 18) {
    echo "Mayor de edad";
} else {
    echo "Menor de edad";
}
```

Y varias condiciones:

```php
if ($nota ⩾ 9) {
    echo "Sobresaliente";
} elseif ($nota ⩾ 7) {
    echo "Notable";
} elseif ($nota ⩾ 5) {
    echo "Aprobado";
} else {
    echo "Suspenso";
}
```

Microejercicio 5 — Calificación

BÁSICO

Crea un programa que reciba una nota y muestre:

- 0–4 → Suspenso
- 5–6 → Aprobado
- 7–8 → Notable
- 9–10 → Sobresaliente

Ver solución explicada y código PHP ▸

10. `match`

PHP moderno proporciona `match` para realizar comparaciones de forma expresiva.

```php
$dia = 1;

$nombreDia = match ($dia) {
    1 ⇒ "Lunes",
    2 ⇒ "Martes",
    3 ⇒ "Miércoles",
    4 ⇒ "Jueves",
    5 ⇒ "Viernes",
    6 ⇒ "Sábado",
    7 ⇒ "Domingo",
    default ⇒ "Día incorrecto"
};

echo $nombreDia;
```

`match` devuelve un valor.

Además, realiza comparaciones estrictas.

Microejercicio 6 — Menú de opciones

BÁSICO

Crea una variable:

```
$opcion = 2;
```

Utiliza `match` para mostrar:

```
1 → Alta de usuario
2 → Listado de usuarios
3 → Cerrar sesión
```

Si la opción no es válida, muestra:

```
Opción incorrecta
```

Ver solución explicada y código PHP ▸

11. Bucles

Los bucles permiten repetir código.

11.1. `while`

```php
$contador = 1;

while ($contador ⩽ 5) {
    echo $contador;
    $contador++;
}
```

11.2. `for`

Cuando conocemos el número de repeticiones podemos utilizar `for`.

```
for ($i = 1; $i ⩽ 5; $i++) {
    echo $i;
}
```

11.3. `foreach`

`foreach` es especialmente importante cuando trabajamos con arrays.

```
$colores = ["rojo", "verde", "azul"];

foreach ($colores as $color) {
    echo $color;
}
```

Microejercicio 7 — Tabla de multiplicar

BÁSICO

Crea una variable:

```
$numero = 7;
```

Utiliza un `for` para mostrar:

```
7 x 1 = 7
7 x 2 = 14
...
7 x 10 = 70
```

Ver solución explicada y código PHP ▸

Bloque 4: Cadenas de Texto (Strings)

APARTADO §12

12. Strings

Una cadena de texto es un `string`.

```
$nombre = "Ana";
```

Podemos obtener su longitud:

```
strlen($nombre);
```

Eliminar espacios iniciales y finales:

```
trim($nombre);
```

Convertir a minúsculas:

```
strtolower($nombre);
```

Convertir a mayúsculas:

```
strtoupper($nombre);
```

Comprobar si contiene un texto:

```
str_contains($nombre, "an");
```

Interpolación

Con comillas dobles:

```
$nombre = "Ana";

echo "Hola $nombre";
```

También podemos utilizar:

```
echo "Hola " . $nombre;
```

Microejercicio 8 — Limpiando nombres

BÁSICO

Crea:

```
$nombre = "   ana garcia   ";
```

Consigue mostrar:

```
Ana Garcia
```

Utiliza `trim()` y las funciones de strings necesarias.

Ver solución explicada y código PHP ▸

Bloque 5: Arrays Indexados, Asociativos y Multidimensionales

APARTADOS §13 A §16

13. Arrays

Los arrays permiten almacenar varios valores.

```
$colores = ["rojo", "verde", "azul"];
```

Podemos acceder a las posiciones:

```
echo $colores[0];
echo $colores[1];
```

Los índices comienzan normalmente en `0`.

13.1. Añadir elementos

```
$colores[] = "amarillo";
```

13.2. `count()`

Podemos saber cuántos elementos contiene:

```
count($colores);
```

13.3. Recorrer un array

```
foreach ($colores as $color) {
    echo $color;
}
```

Microejercicio 9 — Lista de lenguajes BÁSICO

Crea un array con:

```
PHP
JavaScript
HTML
CSS
SQL
```

Recórrelo con `foreach` y muestra cada lenguaje en una línea.

Ver solución explicada y código PHP ▸

14. Arrays asociativos

Un array asociativo utiliza claves en lugar de índices numéricos.

```
$usuario = [
    "nombre" ⇒ "Ana",
    "edad" ⇒ 20,
    "email" ⇒ "ana@example.com"
];
```

Podemos acceder:

```
echo $usuario["nombre"];
echo $usuario["email"];
```

14.1. Modificar valores

```
$usuario["edad"] = 21;
```

14.2. Recorrer claves y valores

```
foreach ($usuario as $clave => $valor) {
    echo "$clave: $valor";
}
```

15. Arrays multidimensionales

Podemos almacenar varios arrays dentro de otro array.

Esto será fundamental para comenzar a representar datos de una aplicación.

```
$productos = [
    [
        "id" => 1,
        "nombre" => "Teclado",
        "precio" => 29.99
    ],
    [
        "id" => 2,
        "nombre" => "Ratón",
        "precio" => 19.99
    ]
];
```

Podemos acceder:

```
echo $productos[0]["nombre"];
```

Y recorrerlos:

```
foreach ($productos as $producto) {
    echo $producto["nombre"];
}
```

Microejercicio 10 — Catálogo

BÁSICO

Crea un catálogo con al menos cinco productos.

Cada producto debe tener:

```
id
nombre
precio
categoria
```

Después muestra todos los productos mediante `foreach`.

Ver solución explicada y código PHP ▸

16. Desestructuración

PHP permite extraer valores de un array de forma sencilla.

```
$colores = ["rojo", "verde", "azul"];

[$primero, $segundo, $tercero] = $colores;
```

Ahora:

```
$primero
```

contiene:

```
rojo
```

Bloque 6: Funciones, Tipado Estricto y PHP 8 Moderno

APARTADOS §17 A §25

17. Funciones

Una función agrupa instrucciones que podemos reutilizar.

```
function saludar(): void
{
    echo "Hola";
}
```

Para ejecutarla:

```
saludar();
```

17.1. Parámetros

```
function saludar(string $nombre): void
{
    echo "Hola $nombre";
}
```

Uso:

```
saludar("Ana");
saludar("Luis");
```

17.2. Retorno

Una función puede devolver un resultado.

```
function sumar(int $a, int $b): int
{
    return $a + $b;
}
```

Uso:

```php
$resultado = sumar(5, 3);

echo $resultado;
```

Microejercicio 11 — Calculadora

BÁSICO

Crea las funciones:

```
sumar()
restar()
multiplicar()
dividir()
```

Todas deben recibir dos números y devolver el resultado.

Ver solución explicada y código PHP ▸

18. Tipado de funciones

En PHP moderno podemos indicar qué tipos esperamos.

```php
function calcularPrecio(float $precio, int $cantidad): float
{
    return $precio * $cantidad;
}
```

Esto hace que el código sea más claro y ayuda a detectar errores.

También podemos utilizar tipos de retorno:

```php
function esMayorDeEdad(int $edad): bool
{
    return $edad ⩾ 18;
}
```

19. `declare(strict_types=1)`

En archivos PHP podemos indicar:

```php
<?php

declare(strict_types=1);
```

Esto debe colocarse al principio del archivo.

Permite que PHP sea más estricto con los tipos escalares al realizar llamadas a funciones.

Ejemplo:

```php
<?php

declare(strict_types=1);

function sumar(int $a, int $b): int
{
    return $a + $b;
}
```

Trabajar con tipos estrictos ayuda a detectar errores antes.

---

20. Tipos modernos

Podemos utilizar tipos como:

```
string
int
float
bool
array
object
mixed
```

También podemos combinar tipos:

```php
function procesar(int|string $valor): string
{
    return (string) $valor;
}
```

Esto se denomina **tipo unión**.

20.1. Nullable

Una variable puede aceptar un valor o `null`:

```
function buscarUsuario(int $id): ?array
{
    // ...
}
```

El `?` significa:

```
array o null
```

También podemos escribir:

```
array|null
```

Microejercicio 12 — Buscar producto

INTERMEDIO

Crea una función:

```
buscarProducto(int $id): ?array
```

que busque un producto dentro de un array.

Debe devolver:

- El producto si existe.
- `null` si no existe.

Ver solución explicada y código PHP ▸

---

21. Operador null coalescing `??`

Es muy habitual que determinados datos no existan.

Podemos hacer:

```
$nombre = $_GET["nombre"] ?? "Invitado";
```

Significa:

> Utiliza `$_GET["nombre"]` si existe y no es `null`; en caso contrario utiliza `"Invitado"`.

También existe:

```
$configuracion ??= [];
```

que asigna un valor solamente si la variable es `null` o no está definida.

---

22. Operador nullsafe `?→`

Cuando trabajamos con objetos podemos tener valores `null`.

PHP permite:

```
$usuario?→getDireccion()?→getCiudad();
```

Si alguna parte es `null`, la expresión devuelve `null` en lugar de producir un error por intentar acceder a un objeto inexistente.

---

23. Operador `⟺`

El operador nave espacial:

```
$a ⟺ $b
```

devuelve:

```
-1  si $a < $b
 0  si $a == $b
 1  si $a > $b
```

Es especialmente útil para funciones de ordenación.

---

24. Argumentos con nombre

Podemos llamar a una función utilizando el nombre de sus parámetros:

```
function crearProducto(
    string $nombre,
    float $precio,
    int $stock
): array {
    return [
        "nombre" ⇒ $nombre,
        "precio" ⇒ $precio,
        "stock" ⇒ $stock
    ];
}
```

Podemos utilizar:

```
crearProducto(
    nombre: "Teclado",
    precio: 29.99,
    stock: 10
);
```

Esto mejora la legibilidad cuando una función tiene varios parámetros.

---

25. Funciones anónimas y arrow functions

Podemos almacenar una función en una variable:

```
$duplicar = function (int $numero): int {
    return $numero * 2;
};
```

También podemos utilizar una función flecha:

```
$duplicar = fn(int $numero): int ⇒ $numero * 2;
```

Las funciones flecha son especialmente útiles al trabajar con arrays.

Bloque 7: Programación Funcional con Arrays y Ordenación

APARTADOS §26 A §30

26. `array_map()`

Permite transformar los elementos de un array.

```
$precios = [10, 20, 30];

$preciosConIVA = array_map(
    fn(float $precio): float ⇒ $precio * 1.21,
    $precios
);
```

Microejercicio 13 — Precios con IVA

INTERMEDIO

Partiendo de:

```
$precios = [10, 20, 50, 100];
```

Utiliza `array_map()` para obtener un nuevo array con el IVA del 21 % incluido.

Ver solución explicada y código PHP ▸

27. `array_filter()`

Permite quedarse solamente con los elementos que cumplen una condición.

```
$precios = [10, 20, 50, 100];

$caros = array_filter(
    $precios,
    fn(float $precio): bool ⇒ $precio ⩾ 50
);
```

Microejercicio 14 — Filtrar productos

INTERMEDIO

Crea un array de productos.

Utiliza `array_filter()` para obtener únicamente aquellos cuyo precio sea superior a 50 €.

Ver solución explicada y código PHP ▸

28. `array_reduce()`

Permite reducir un array a un único resultado.

Por ejemplo, sumar precios:

```php
$precios = [10, 20, 30];

$total = array_reduce(
    $precios,
    fn(float $total, float $precio): float ⇒ $total + $precio,
    0
);
```

Microejercicio 15 — Total del carrito

INTERMEDIO

Dado:

```php
$precios = [10, 25, 30, 15];
```

Calcula el total utilizando `array_reduce()`.

Después añade una función que calcule el IVA.

Ver solución explicada y código PHP ▸

29. `array_column()`

Si tenemos:

```
$productos = [
    ["nombre" ⇒ "Teclado", "precio" ⇒ 30],
    ["nombre" ⇒ "Ratón", "precio" ⇒ 20],
    ["nombre" ⇒ "Monitor", "precio" ⇒ 200]
];
```

Podemos obtener todos los precios:

```
$precios = array_column($productos, "precio");
```

---

30. Ordenación

PHP proporciona distintas funciones para ordenar arrays.

Por ejemplo:

```
sort($precios);
```

Para arrays asociativos podemos utilizar:

```
asort();
ksort();
```

Cuando necesitamos una ordenación personalizada podemos utilizar:

```
usort();
```

Ejemplo:

```
usort(
    $productos,
    fn(array $a, array $b): int ⇒
        $a["precio"] ⟺ $b["precio"]
);
```

Microejercicio 16 — Ordenar catálogo

INTERMEDIO

Ordena un catálogo de productos:

- De menor a mayor precio.
- De mayor a menor precio.
- Alfabéticamente por nombre.

  Utiliza `usort()`.

Ver solución explicada y código PHP ▸

Bloque 8: Modularidad, Ámbitos y Excepciones

APARTADOS §31 A §33

31. Ámbito de las variables

Una variable definida fuera de una función no pertenece automáticamente al ámbito interno de la función.

```
$nombre = "Ana";

function mostrarNombre(): void
{
    // $nombre no está disponible directamente aquí.
}
```

La opción preferida suele ser pasar los datos como parámetros:

```
function mostrarNombre(string $nombre): void
{
    echo $nombre;
}
```

Es más claro y facilita las pruebas.

Debemos evitar abusar de `global`.

---

32. Separar código con `require`

A medida que los programas crecen, no debemos meter todo en un único archivo.

Podemos tener:

```
src/
    funciones.php
public/
    index.php
```

En `funciones.php`:

```php
<?php

function sumar(int $a, int $b): int
{
    return $a + $b;
}
```

En `index.php`:

```php
<?php

require __DIR__ . '/../src/funciones.php';

echo sumar(5, 3);
```

`require` incluye el archivo.

Para archivos necesarios para que el programa funcione, `require` suele ser la opción adecuada.

Microejercicio 17 — Primer proyecto dividido

INTERMEDIO

Crea:

```
proyecto/
├── public/
│   └── index.php
└── src/
    └── funciones.php
```

Mueve las funciones de la calculadora a `funciones.php`.

Cárgalas desde `index.php`.

Ver solución explicada y código PHP ▸

---

33. Excepciones

Una excepción permite representar un problema que debe gestionarse.

```
throw new Exception("Ha ocurrido un error");
```

Podemos capturarla:

```
try {
    // Código que puede fallar
} catch (Exception $e) {
    echo $e→getMessage();
}
```

Ejemplo:

```
function dividir(float $a, float $b): float
{
    if ($b === 0.0) {
        throw new InvalidArgumentException(
            "No se puede dividir entre cero"
        );
    }

    return $a / $b;
}
```

Microejercicio 18 — División segura

INTERMEDIO

Modifica la función `dividir()` para que lance una excepción si el segundo parámetro es `0`.

Captura la excepción y muestra un mensaje adecuado al usuario.

Ver solución explicada y código PHP ▸

Bloque 9: Integración de PHP en HTML

APARTADO §34

34. PHP y HTML

PHP suele utilizarse junto con HTML.

```
<!DOCTYPE html>
<html lang="es">
<head>
    <meta charset="UTF-8">
    <title>Mi página</title>
</head>
<body>

<h1>
    <?php echo "Hola desde PHP"; ?>
</h1>

</body>
</html>
```

Existe una sintaxis abreviada especialmente útil para mostrar valores:

```
<?= $nombre ?>
```

Por ejemplo:

```
<h1>Hola <?= htmlspecialchars($nombre) ?></h1>
```

Bloque 10: Formularios, GET, POST y Validación Segura

APARTADOS §35 A §46

35. El modelo petición-respuesta

A partir de ahora es importante entender este concepto.

Cada vez que un navegador solicita una página:

```
Navegador
   |
   | GET /productos.php
   ▼
Servidor
   |
   | ejecuta PHP
   ▼
Respuesta HTML
   |
   ▼
Navegador
```

PHP procesa la petición y genera una respuesta.

Los datos enviados por el navegador están disponibles mediante determinadas variables y mecanismos.

---

36. Formularios HTML

Un formulario permite enviar información al servidor.

Ejemplo:

```
<form method="get" action="buscar.php">

    <label for="q">Buscar:</label>

    <input
        type="text"
        id="q"
        name="q"
    >

    <button type="submit">
        Buscar
    </button>

</form>
```

El atributo:

```
name="q"
```

es importante porque será el nombre con el que PHP recibirá el dato.

---

37. GET

Si utilizamos:

```
<form method="get">
```

los datos forman parte de la URL.

Por ejemplo:

```
buscar.php?q=php
```

PHP puede acceder al valor mediante:

```
$q = $_GET["q"] ?? "";
```

GET suele ser adecuado para operaciones como:

- búsquedas;
- filtros;
- navegación;
- parámetros que pueden aparecer en la URL.

Microejercicio 19 — Buscador

BÁSICO

Crea un formulario:

```
Buscar producto: [___________] [Buscar]
```

Cuando el usuario lo envíe mediante GET, muestra:

```
Has buscado: teclado
```

Utiliza `$_GET`.

Ver solución explicada y código PHP ▸

---

38. POST

Con:

```
<form method="post">
```

los datos se envían en el cuerpo de la petición HTTP.

PHP puede acceder a ellos mediante:

```
$_POST
```

Ejemplo:

```
$nombre = $_POST["nombre"] ?? "";
```

POST es habitual para operaciones como:

- crear datos;
- modificar datos;
- iniciar sesión;
- enviar formularios;
- subir archivos.

Importante:

> POST no cifra los datos por sí mismo. La protección frente a observación durante el transporte la proporciona HTTPS.

---

39. `$_SERVER`

PHP proporciona información sobre la petición mediante `$_SERVER`.

Por ejemplo:

```
$_SERVER["REQUEST_METHOD"]
```

permite conocer si la petición es:

```
GET
POST
```

Podemos hacer:

```
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    // Procesamos el formulario
}
```

Esto permite que una misma página muestre el formulario y procese su envío.

---

40. No utilizar `$_REQUEST`

Aunque existe:

```
$_REQUEST
```

no es recomendable utilizarlo para trabajar con entradas de usuario.

Es preferible saber explícitamente de dónde procede el dato:

```
$_GET
```

o:

```
$_POST
```

Esto hace el código más claro y reduce errores de seguridad.

---

41. Validación

Nunca debemos confiar directamente en los datos recibidos.

El navegador puede enviar cualquier cosa.

Por ejemplo:

```
$edad = $_POST["edad"] ?? "";
```

No significa que `$edad` sea realmente una edad válida.

Debemos:

1. Obtener el dato.
2. Normalizarlo cuando proceda.
3. Validarlo.
4. Rechazar valores incorrectos.

5. Utilizarlo solamente después de comprobarlo.

---

42. `filter_input()`

PHP dispone de:

```
filter_input()
```

Por ejemplo:

```
$edad = filter_input(
    INPUT_POST,
    "edad",
    FILTER_VALIDATE_INT
);
```

Si el valor no es un entero válido, podemos detectar el error.

Para un email:

```
$email = filter_input(
    INPUT_POST,
    "email",
    FILTER_VALIDATE_EMAIL
);
```

---

43. Validación y reglas de negocio

No toda validación se resuelve con un filtro.

Por ejemplo:

```
$edad = filter_input(
    INPUT_POST,
    "edad",
    FILTER_VALIDATE_INT
);

if ($edad === false || $edad < 18) {
    $errores["edad"] = "Debes tener al menos 18 años.";
}
```

Aquí combinamos:

- validación del tipo;

- regla de negocio.

Microejercicio 20 — Formulario de registro

INTERMEDIO

Crea un formulario con:

```
Nombre
Email
Edad
```

Valida:

- Nombre obligatorio.
- Email válido.
- Edad entera.
- Edad mínima de 18 años.

  Guarda los errores en un array:

  ```
  $errores = [];
  ```

Ver solución explicada y código PHP ▸

---

44. Repoblar formularios

Si un formulario tiene errores, no queremos que el usuario tenga que escribir todo de nuevo.

Podemos guardar los valores:

```
$nombre = $_POST["nombre"] ?? "";
```

Y posteriormente:

```
<input
    type="text"
    name="nombre"
    value="<?= htmlspecialchars($nombre) ?>"
>
```

---

45. Escapado y XSS

Uno de los problemas de seguridad más importantes en aplicaciones web es **XSS — Cross-Site Scripting**.

Imaginemos que un usuario introduce:

```
<script>alert("Hacked")</script>
```

Si mostramos directamente ese contenido:

```
echo $nombre;
```

podríamos estar permitiendo que el navegador interprete HTML o JavaScript enviado por el usuario.

Por eso, cuando mostramos datos controlados por el usuario dentro de HTML, utilizamos:

```
htmlspecialchars()
```

Ejemplo:

```
echo htmlspecialchars(
    $nombre,
    ENT_QUOTES,
    "UTF-8"
);
```

En una plantilla:

```
<?= htmlspecialchars($nombre, ENT_QUOTES, "UTF-8") ?>
```

Regla fundamental

> **Entrada de usuario = no confiable.**

Debemos validar los datos para comprobar que cumplen nuestras reglas.

Y debemos escaparlos según el contexto cuando los mostramos.

Microejercicio 21 — Prueba XSS

INTERMEDIO

Crea un formulario que permita introducir un nombre.

Introduce como nombre:

```
<script>alert("Hola")</script>
```

Primero muestra el valor directamente y observa el resultado.

Después utiliza:

```
htmlspecialchars()
```

y comprueba la diferencia.

Ver solución explicada y código PHP ▸

46. Errores del formulario

Es habitual mantener un array:

```php
$errores = [];
```

Por ejemplo:

```php
if ($nombre === "") {
    $errores["nombre"] = "El nombre es obligatorio.";
}
```

Después podemos mostrar:

```php
<?php if (isset($errores["nombre"])): ?>

    <p>
        <?= htmlspecialchars($errores["nombre"]) ?>
    </p>

<?php endif; ?>
```

De esta forma podemos asociar errores a campos concretos.

Bloque 11: Subida Segura de Archivos ($_FILES)

APARTADOS §47 A §52

47. Subida de archivos

HTML permite subir archivos:

```
<form
    method="post"
    enctype="multipart/form-data"
>
    <input type="file" name="imagen">

    <button type="submit">
        Subir
    </button>
</form>
```

Es imprescindible:

```
enctype="multipart/form-data"
```

48. `$_FILES`

PHP proporciona información sobre el archivo mediante:

```
$_FILES
```

Por ejemplo:

```
$archivo = $_FILES["imagen"];
```

Contiene información como:

```
$archivo["name"];
$archivo["type"];
$archivo["tmp_name"];
$archivo["error"];
$archivo["size"];
```

49. No confiar en la extensión

No debemos decidir que un archivo es seguro simplemente porque se llama:

```
foto.jpg
```

Un usuario puede manipular el nombre o el contenido.

Para detectar el tipo real podemos utilizar:

```
$finfo = new finfo(FILEINFO_MIME_TYPE);

$mime = $finfo→file($archivo["tmp_name"]);
```

Después podemos permitir únicamente determinados tipos.

Por ejemplo:

```
$permitidos = [
    "image/jpeg",
    "image/png",
    "image/webp"
];
```

---

50. Nombre seguro del archivo

No deberíamos utilizar directamente:

```
$archivo["name"]
```

como nombre final.

Podemos generar un nombre aleatorio:

```
$nombreSeguro = bin2hex(random_bytes(16));
```

Después añadiremos una extensión que nosotros hayamos determinado a partir del tipo permitido.

---

51. Mover el archivo

Una vez validado:

```
move_uploaded_file(
    $archivo["tmp_name"],
    $rutaDestino
);
```

Es importante comprobar siempre:

- error de subida;
- tamaño;
- tipo MIME;
- extensión permitida;
- nombre generado;
- ubicación de almacenamiento.

Microejercicio 22 — Subida segura

INTERMEDIO

Crea un formulario para subir una imagen.

Acepta solamente:

```
JPEG
PNG
WebP
```

Rechaza:

- archivos demasiado grandes;
- tipos no permitidos;
- errores de subida.

  Genera un nombre aleatorio para guardar el archivo.

Ver solución explicada y código PHP ▸

---

52. Límites de PHP

La subida de archivos también depende de la configuración de PHP.

Algunos parámetros importantes son:

```
upload_max_filesize
post_max_size
max_file_uploads
```

No basta con validar en el programa: el servidor también tiene sus propios límites.

Bloque 12: Gestión del Estado: Cookies y Sesiones

APARTADOS §53 A §63

53. HTTP no mantiene estado

HTTP es, conceptualmente, un protocolo **sin estado**.

El servidor procesa una petición:

```
Petición 1 → Respuesta
```

y después otra:

```
Petición 2 → Respuesta
```

No debemos asumir que PHP recuerda automáticamente que ambas peticiones pertenecen al mismo usuario.

Pero una aplicación necesita recordar cosas:

- usuario autenticado;
- carrito;
- preferencias;
- idioma;
- mensajes temporales.

Para ello tenemos mecanismos de estado como:

- cookies;
- sesiones.

---

54. Cookies

Una cookie es un pequeño dato que el servidor puede solicitar al navegador que almacene.

Podemos crear una:

```
setcookie(
    "tema",
    "oscuro",
    time() + 86400
);
```

Después el navegador enviará la cookie en determinadas peticiones.

PHP puede acceder a ella mediante:

```
$_COOKIE["tema"] ?? "claro";
```

---

55. Atributos de seguridad de las cookies

Cuando trabajamos con información sensible debemos prestar atención a:

`Secure`

La cookie solamente se enviará mediante HTTPS.

`HttpOnly`

Impide que JavaScript pueda acceder directamente a la cookie mediante `document.cookie`.

`SameSite`

Controla cuándo el navegador envía la cookie en contextos relacionados con otros sitios.

Ejemplo:

```
setcookie(
    "preferencia",
    "oscuro",
    [
        "expires" ⇒ time() + 86400,
        "path" ⇒ "/",
        "secure" ⇒ true,
        "httponly" ⇒ true,
        "samesite" ⇒ "Lax"
    ]
);
```

---

56. Sesiones

Las sesiones permiten almacenar información asociada a un usuario entre diferentes peticiones.

Comenzamos una sesión:

```
session_start();
```

Después podemos utilizar:

```
$_SESSION
```

Por ejemplo:

```
$_SESSION["contador"] = 1;
```

En otra petición podremos recuperar:

```
echo $_SESSION["contador"];
```

---

57. El identificador de sesión

PHP necesita identificar qué sesión corresponde a cada navegador.

Habitualmente utiliza una cookie como:

```
PHPSESSID
```

El navegador envía el identificador y PHP recupera los datos de sesión correspondientes.

Conceptualmente:

```
Navegador
   |
   | PHPSESSID=abc123
   ▼
Servidor
   |
   | busca sesión abc123
   ▼
Datos de sesión
```

---

58. Ciclo de vida de una sesión

Un proceso típico es:

```
session_start();
```

Después:

```
$_SESSION["usuario_id"] = 25;
```

Para eliminar un dato:

```
unset($_SESSION["usuario_id"]);
```

Para destruir la sesión:

```
session_destroy();
```

En un logout completo también conviene limpiar los datos de sesión y gestionar correctamente la cookie de sesión.

Microejercicio 23 — Contador de visitas

BÁSICO

Crea una página que incremente un contador de sesión cada vez que se recarga.

Debe mostrar:

```
Has visitado esta página 7 veces en esta sesión.
```

Ver solución explicada y código PHP ▸

---

59. Carrito de compra con sesiones

Un carrito puede almacenarse en:

```
$_SESSION["carrito"]
```

Por ejemplo:

```
$_SESSION["carrito"] = [
    1 ⇒ 2,
    5 ⇒ 1
];
```

Podemos interpretar:

```
producto 1 → cantidad 2
producto 5 → cantidad 1
```

Esto será muy útil para nuestro proyecto de catálogo.

Microejercicio 24 — Carrito con sesiones

INTERMEDIO

Crea un catálogo de productos y permite:

- añadir un producto;
- aumentar su cantidad;
- eliminarlo;
- consultar el carrito.

  El carrito debe sobrevivir entre diferentes peticiones utilizando `$_SESSION`.

Ver solución explicada y código PHP ▸

---

60. Sesiones y seguridad

Una sesión puede ser robada si un atacante consigue su identificador.

Por ello debemos:

- utilizar HTTPS;
- proteger la cookie de sesión;
- regenerar el identificador cuando cambia el nivel de privilegio;
- cerrar sesiones correctamente;
- establecer tiempos de expiración razonables.

---

61. Regeneración de sesión

Después de autenticar correctamente a un usuario:

```
session_regenerate_id(true);
```

Esto ayuda a prevenir ataques de **session fixation**.

La idea es que el identificador de sesión utilizado antes del login no siga siendo el mismo después de autenticarse.

También puede ser apropiado regenerar la sesión cuando se produce un cambio importante de privilegios.

---

62. Expiración por inactividad

Podemos almacenar la última actividad:

```
$_SESSION["ultima_actividad"] = time();
```

En cada petición comprobamos:

```
$ahora = time();

if (
    isset($_SESSION["ultima_actividad"])
    && $ahora - $_SESSION["ultima_actividad"] > 1800
) {
    // Expirar sesión
}
```

En este ejemplo:

```
1800 segundos = 30 minutos
```

63. Tiempo de vida absoluto

Además de la inactividad, podemos establecer una duración máxima absoluta desde el inicio de la sesión, independientemente de si el usuario ha estado activo o no.

Por ejemplo, al iniciar sesión:

```
$_SESSION["inicio_sesion"] = time();
```

Después, en cada comprobación:

```php
$duracionMaxima = 7200; // 2 horas (en segundos)

if (
    isset($_SESSION["inicio_sesion"])
    && (time() - $_SESSION["inicio_sesion"] > $duracionMaxima)
) {
    // La sesión ha superado el tiempo máximo permitido
    $_SESSION = [];
    session_destroy();
    header("Location: login.php?mensaje=sesion_expirada");
    exit;
}
```

Combinar el control de inactividad con un tiempo de vida absoluto es una práctica estándar en aplicaciones seguras.

Bloque 13: Hashing de Contraseñas y Criptografía — APARTADO §64

64. Hashing de contraseñas y criptografía

Nunca debemos almacenar contraseñas en texto plano ni utilizar algoritmos obsoletos como **MD5** o **SHA-1**, los cuales son fácilmente vulnerables a ataques de diccionario y tablas *rainbow*.

Tampoco debemos inventar esquemas propios de cifrado. En seguridad web, las contraseñas se almacenan mediante **funciones de derivación de claves seguras y lentas (hashing unidireccional)**.

PHP proporciona funciones nativas especializadas y robustas:

- `password_hash()`
- `password_verify()`
- `password_needs_rehash()`

64.1. Generar un hash seguro con `password_hash()`

```php
$clavePlana = "secreto123";

$hash = password_hash($clavePlana, PASSWORD_BCRYPT);
```

Por defecto, PHP utiliza el algoritmo **Bcrypt** (`PASSWORD_BCRYPT`) o el algoritmo por defecto del sistema (`PASSWORD_DEFAULT`), generando automáticamente una **sal (*salt*) criptográfica única** para cada hash y aplicando un factor de coste computacional.

El resultado es una cadena de 60 caracteres:

```
$2y$10$wK1FmNf/7Yq2aJ0sF.2jtuVlM/u6T6nFzS2wV0n3T0F3yL1a4G2k2
```

Incluso si dos usuarios tienen la misma contraseña, sus hashes resultantes serán completamente diferentes gracias a la sal única generada de forma transparente.

Podemos ajustar el factor de coste (*cost*) para aumentar el tiempo de cómputo y dificultar la fuerza bruta:

```php
$opciones = [
    'cost' => 12,
];

$hash = password_hash($clavePlana, PASSWORD_BCRYPT, $opciones);
```

64.2. Verificar una contraseña con `password_verify()`

Para comprobar si la contraseña introducida por el usuario coincide con el hash guardado:

```php
$claveIntroducida = "secreto123";
$hashAlmacenado = '$2y$10$wK1FmNf/7Yq2aJ0sF.2jtuVlM/u6T6nFzS2wV0n3T0F3yL1a4G2k2';

if (password_verify($claveIntroducida, $hashAlmacenado)) {
    echo "Contraseña correcta. Acceso concedido.";
} else {
    echo "Contraseña incorrecta. Acceso denegado.";
}
```

`password_verify()` extrae automáticamente la sal y los parámetros del hash almacenado, calcula el nuevo hash de la clave introducida y los compara en **tiempo constante**, protegiendo la aplicación frente a ataques de temporización (*timing attacks*).

Microejercicio 25 — Hashing y verificación de contraseñas

BÁSICO

- Crea una variable con una contraseña en texto plano.
- Genera su hash utilizando `password_hash()` con `PASSWORD_BCRYPT`.
- Muestra el hash generado y comprueba su longitud con `strlen()`.
- Utiliza `password_verify()` para comprobar si la contraseña correcta valida el hash.
- Prueba a pasar una contraseña errónea y comprueba que `password_verify()` devuelve `false`.

Ver solución explicada y código PHP ▸

Bloque 14: Autenticación, Login Seguro y Logout

APARTADOS §65 Y §66

65. Autenticación de usuarios: Flujo de Login seguro

El proceso de autenticación comprueba la identidad declarada por el usuario (normalmente mediante email/usuario y contraseña).

Un flujo de login seguro en PHP debe seguir estos pasos:

1. Iniciar sesión con `session_start()`.
2. Verificar que la petición sea `POST`.
3. Validar y sanear los campos de entrada.
4. Buscar al usuario en la base de datos o almacén de usuarios.
5. Comprobar la contraseña mediante `password_verify()`.
6. Si las credenciales son válidas:

Regenerar el identificador de sesión con `session_regenerate_id(true)` *para evitar* Session Fixation*.

* Guardar los datos esenciales del usuario en `$_SESSION` (por ejemplo, `id`, `nombre` y `rol`).

* Redirigir a la zona privada.

1. Si no son válidas, mostrar un mensaje de error genérico (ejemplo: *"Usuario o contraseña incorrectos"* para no dar pistas a atacantes sobre qué campo falló).

Ejemplo de procesamiento de login:

```php
session_start();

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $email = filter_input(INPUT_POST, "email", FILTER_VALIDATE_EMAIL);
    $password = $_POST["password"] ?? "";

    // Simulación de búsqueda en catálogo de usuarios
    $usuario = buscarUsuarioPorEmail($email);

    if ($usuario && password_verify($password, $usuario["password_hash"])) {
        // 1. Regeneramos el ID de sesión
        session_regenerate_id(true);

        // 2. Guardamos datos en sesión
        $_SESSION["usuario_id"] = $usuario["id"];
        $_SESSION["nombre"] = $usuario["nombre"];
        $_SESSION["rol"] = $usuario["rol"];
        $_SESSION["ultimo_acceso"] = time();

        // 3. Redirigimos al panel privado
        header("Location: dashboard.php");
        exit;
    } else {
        $error = "Credenciales incorrectas.";
    }
}
```

---

66. Cierre de sesión seguro (*Logout*)

Para cerrar la sesión de forma completamente segura no basta con llamar a `session_destroy()`. El protocolo recomendado de 4 pasos es:

```php
<?php

declare(strict_types=1);

session_start();

// 1. Vaciar el array de variables de sesión
$_SESSION = [];

// 2. Si se utilizan cookies de sesión, forzar su expiración en el navegador
if (ini_get("session.use_cookies")) {
    $params = session_get_cookie_params();
    setcookie(
        session_name(),
        '',
        time() - 42000,
        $params["path"],
        $params["domain"],
        $params["secure"],
        $params["httponly"]
    );
}

// 3. Destruir la sesión en el almacenamiento del servidor
session_destroy();

// 4. Redirigir al usuario al login o página principal
header("Location: login.php?mensaje=sesion_cerrada");
exit;
```

Microejercicio 26 — Login y Logout completos

INTERMEDIO

Crea un sistema mínimo de login y logout:

- Un array simulado con 2 usuarios y sus contraseñas hasheadas con `password_hash()`.
- Un formulario de login en `login.php`.
- Un archivo `dashboard.php` que muestre *"Bienvenido, [nombre]"* y un enlace a `logout.php`.
- El script `logout.php` debe vaciar la sesión, destruir la cookie de sesión y redirigir a `login.php`.

Ver solución explicada y código PHP ▸

Bloque 15: Control de Acceso por Roles (RBAC) APARTADO §67

67. Control de acceso y autorización por roles (RBAC)

Una vez que un usuario está autenticado, la **autorización** determina a qué partes de la aplicación tiene acceso.

67.1. Proteger una página privada

En cualquier página que requiera estar autenticado, colocamos un control previo:

```
<?php

declare(strict_types=1);

session_start();

// Comprobar si el usuario ha iniciado sesión
if (!isset($_SESSION["usuario_id"])) {
    // Redirigir al formulario de login
    header("Location: login.php?error=acceso_denegado");
    exit;
}
```

¡Muy importante! Siempre que ejecutemos `header("Location: ...")` debemos incluir inmediatamente un `exit;` o `die();` a continuación. De lo contrario, el intérprete de PHP continuará ejecutando las líneas inferiores del script antes de que el navegador cambie de página.

67.2. Control de acceso por roles (*Role-Based Access Control*)

Si un script solo puede ser ejecutado por administradores:

```php
<?php

declare(strict_types=1);

session_start();

// 1. Debe estar autenticado
if (!isset($_SESSION["usuario_id"])) {
    header("Location: login.php");
    exit;
}

// 2. Debe tener rol de administrador
if (($_SESSION["rol"] ?? "") !== "admin") {
    http_response_code(403);
    echo "<h1>403 Prohibido</h1><p>No tienes permisos suficientes para acceder a esta sección.</p>"
    exit;
}
```

Podemos encapsular estas comprobaciones en funciones auxiliares de autenticación (`requiereAutenticacion()` y `requiereRol('admin')`) para reutilizarlas en toda la aplicación.

Microejercicio 27 — Zona privada por roles

INTERMEDIO

Crea una página `admin.php` que:

- Compruebe si el usuario está autenticado. Si no, lo redirija a `login.php` .
- Compruebe si el rol es `"admin"` . Si el rol es `"usuario"` , muestre un mensaje de error 403.
- Si el rol es `"admin"` , muestre un panel de administración con un listado simulado de usuarios.

Ver solución explicada y código PHP ▸

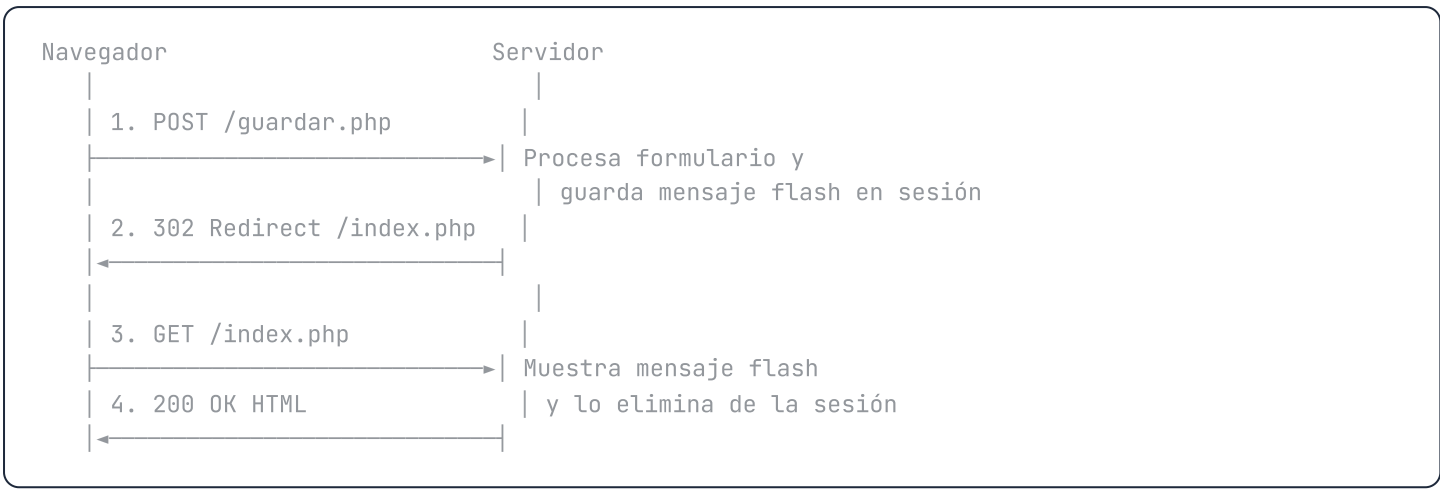
Bloque 16: Mensajes Flash y Patrón PRG

APARTADO §68

68. Mensajes temporales (*Flash Messages*) y patrón PRG

Cuando un usuario envía un formulario por `POST` (por ejemplo, guardar un producto), si dejamos que la respuesta cargue directamente la misma página, ocurrirá un problema común: si el usuario pulsa **F5 (Recargar)**, el navegador le advertirá si desea volver a enviar los datos, pudiendo duplicar la compra o el registro.

Para solucionar esto se utiliza el **patrón PRG (*Post-Redirect-Get*)**:



68.1. Implementación de mensajes Flash con `$_SESSION`

Al procesar la acción:

```php
// Procesamiento del formulario completado con éxito
$_SESSION["flash"] = [
    "tipo" ⇒ "exito",
    "mensaje" ⇒ "El producto se ha guardado correctamente."
];

header("Location: productos.php");
exit;
```

En la vista de destino (`productos.php`):

```php
session_start();

$flash = $_SESSION["flash"] ?? null;
unset($_SESSION["flash"]); // Se elimina para que solo se muestre una vez
?>

<?php if ($flash): ?>
    <div class="alerta alerta—<?= htmlspecialchars($flash['tipo']) ?>">
        <?= htmlspecialchars($flash['mensaje']) ?>
    </div>
<?php endif; ?>
```

Microejercicio 28 — Mensaje Flash y patrón PRG

BÁSICO

Crea un formulario simple de contacto o suscripción:

- El formulario envía datos por `POST` a `procesar.php`.
- `procesar.php` valida el email, guarda un mensaje en `$_SESSION["mensaje_flash"]` y redirige con `header("Location: index.php")`.
- `index.php` muestra el mensaje de confirmación si existe y lo elimina para que desaparezca al recargar la página.

Ver solución explicada y código PHP ▸

Bloque 17: Organización de Proyectos y Resumen

APARTADOS §69 Y §70

69. Organización y arquitectura básica de un proyecto en PHP puro

A medida que el código crece, mezclar en un solo archivo consultas, lógica de negocio y etiquetas HTML genera código desordenado e inmantenible (*código espagueti*).

Una estructura profesional básica en PHP puro antes de aprender un framework como Laravel organiza el proyecto así:

```
mi-proyecto/
├── config/
│   └── app.php             # Constantes y configuración global
├── src/
│   ├── helpers.php          # Funciones auxiliares (saneamiento, redirección, flash)
│   └── auth.php             # Funciones de autenticación y verificación de roles
├── views/
│   ├── layout/
│   │   ├── header.php       # Cabecera HTML y menú de navegación
│   │   └── footer.php       # Cierre de etiquetas y scripts
│   ├── login.view.php       # Vista del formulario de login
│   ├── catalogo.view.php    # Vista del catálogo
│   └── dashboard.view.php   # Vista privada
└── public/
    ├── index.php            # Punto de entrada / Enrutador básico
    ├── login.php            # Controlador de login
    ├── logout.php           # Controlador de logout
    ├── admin.php            # Controlador de zona administrativa
    └── assets/              # Archivos estáticos (CSS, JS, imágenes)
        └── css/style.css
```

Reglas de oro:

1. **Separación de responsabilidades:** La lógica de proceso se realiza arriba en el archivo controlador y las vistas se limitan a mostrar variables mediante sintaxis alternativa (`...`).
2. **Escapar siempre la salida:** Toda variable mostrada en HTML debe pasar por `htmlspecialchars()` .
3. **Punto de redirección:** Siempre acompañar `header("Location: ...")` de `exit;` .

---

70. Resumen de la Unidad 2 y Buenas Prácticas

Área	Buena Práctica Recomendada	Práctica Insegura / Obsoleta
Tipado y Archivos	`declare(strict_types=1);` y tipos explícitos en funciones.	Tipado débil sin declarar y conversión implícita descontrolada.
Control de Flujo	Expresiones `match` con comparaciones estrictas (`===`).	Abuso de `switch` con comparaciones débiles (`==`) y olvido de `break` .
Entrada de Datos	`filter_input(INPUT_POST, ...)` y comprobación de tipo y rango.	Acceso directo sin comprobar o uso de `$_REQUEST` .
Seguridad XSS	`htmlspecialchars($var, ENT_QUOTES, 'UTF-8')` en toda salida a HTML.	`echo $_POST['nombre'];` directo sin escapar.
Subida de Ficheros	Validar tipo MIME con `finfo` , tamaño y renombrar con nombre aleatorio.	Confiar en la extensión o en `$_FILES['archivo']['type']` .
Cookies	Flags `HttpOnly` , `Secure` , `SameSite=Lax/Strict` .	Cookies sin flags de seguridad accesibles desde JavaScript.
Sesiones	`session_regenerate_id(true)` tras el login y expiración por inactividad.	Mantener el mismo ID de sesión permanentemente (*Session Fixation*).
Contraseñas	`password_hash()` con Bcrypt o Argon2id y `password_verify()` .	MD5, SHA-1, SHA-256 plano o contraseñas en texto claro.
Redirecciones	`header('Location: ...'); exit;`	`header('Location: ...');` sin `exit` (continúa ejecutando código).
